¿Cuál es la diferencia entre una lista y una tupla en Python?

La principal diferencia entre ellas es que las listas son mutables y se definen con [] y las tuplas son inmutables y se definen con (), para saber cual debemos usar de las dos, lo primero que debemos de preguntarnos es para que vamos a utilizarlo, si los datos cambian o no.

Un ejemplo de lista seria:

```
lista = [1, 2, 3]
lista[0] = 0

print(lista)
```

Esto nos devolverá [0, 2, 3] cambiando la primera posición de la lista

Por el contrario, con la tupla:

```
tupla = (1, 2, 3)
tupla[0] = 0

print(tupla)
```

Esto nos devolverá un error ya que las tuplas no aceptan ese tipo de cambios.

Un método para agregar datos a una tupla seria:

```
tupla_nueva = tupla + (4,)

print(tupla_nueva)
```

La salida seria (1, 2, 3, 4)

También podemos pasar la lista a tupla y la tupla a lista, aunque hay que tener cuidado porque se tratan de diferente forma:

Para pasar la lista a tupla usaremos: lista = list(tupla)

Para pasar de tupla a lista usaremos: tupla = tuple(lista)

¿Cuál es el orden de las operaciones?

El orden es el mismo que usan las matemáticas de toda la vida. Siguiendo este orden:

1. Paréntesis ()
2. Exponenciales **
3. Multiplicar * Dividir \
4. Sumar + Restar –

Primero resolvemos el interior de los paréntesis, después los exponenciales o potencias, seguido de multiplicaciones y divisiones y por último sumas y restas.

Ejemplo

operaciones = 2 + 3 * (4 - 1) ** 2 =

Primero resolvemos paréntesis	2 + 3 * 3 ** 2
Después exponenciales	2 + 3 * 9
Mas tarde división y multiplicación	2 + 27
Por último, sumas y restas	29

Por lo tanto se ponemos print(operaciones) el resultado será 29

Esto es muy importante porque si no seguimos esas reglas puede darnos problemas.

¿Qué es un diccionario Python?

Un diccionario es una forma de almacenar datos en pares de Clave\Valor. Se define con {} en ellas se almacena una clave y un valor por línea. La clave es algo único no se puede repetir ya sean letras o números, como un DNI no puede haber dos personas con el mismo numero. Y a su lado el valor asociado a esa llave. Por ejemplo:

```
persona = {
    "nombre": "Juan",
    "edad": 64,
    "ciudad": "Madrid"
}
```

En el ejemplo las claves serian la parte izquierda dentro de {} y su valor la parte derecha.

Su ejecutásemos print(persona["ciudad"]) nos devolvería Madrid ya que estaríamos pidiendo que dentro del diccionario persona nos de el valor de la clave ciudad.

Podemos agregar/eliminar y modificar los elementos del diccionario.

Ejemplo:

del persona["ciudad"]	Eliminará la clave ciudad
persona[“pais”] = España	Agregará una nueva clave o la sobrescribirá si ya existía

También tenemos la opción de anidar varios diccionarios.

Ejemplo:

```
personas = {
    "madre": {
    "Nombre": "Maite”,
    "Apellido": "Ramirez",
    "Edad": 56
  },
```

```
    "padre": {
      "Nombre": "Pedro",
      "Apellido": "Gongora",
      "Edad": 51
    }
}
```

¿Cuál es la diferencia entre el método ordenado y la función de ordenación?

Ambas sirven para ordenar pero tienen sus diferencias. El método de ordenación ordena los datos de forma permanente, en cambio la función de ordenación te devuelve los datos ordenados sin modificar el orden principal, así podríamos llamar a los datos con el orden que se les dio originalmente. Por ejemplo:

```
metodo_ordenado = [5, 2, 9, 1]
metodo_ordenado.sort()

print(metodo_ordenado)
```

Nos devuelve los datos ordenados 1, 2, 5, 9 también podríamos utilizar metodo_ordenado.sort(reverse=true) para mostrarnos los datos de forma inversa.

```
funcion_ordenacion = [5, 2, 9, 1]
función_de_ordenacion = sorted(funcion_ordenacion)

print(funcion_ordenacion)
```

Nos devuelve el mismo orden principal 5, 2, 9, 1

```
print(función_de_ordenacion)
```

Nos devuelve los datos ordenados 1, 2, 5, 9.

Sort() es utilizado con tipos de datos mutables como listas, mientas que sorted()es usado con tanto con listas como tuplas.

¿Qué es un operador de reasignación?

Nos permite actualizar el valor de una variable de forma abreviada y fácil. Esto nos ayuda a mantener un código más ordenado y limpio, además de ahorrarnos repetir datos en el código.

Ejemplo: si tenemos un valor de 5 y quisiésemos sumar 5 lo primero que pensamos es en

```
x = 5
x = x + 5
```

Pero podemos abreviarlo con un operador de reasignación

```
x = 5
x += 5
```

Si llamamos a x nos devolverá 10, por lo tanto hemos actualizado el valor de x que ya no será 5

Hay varios operadores que podemos usar y todos se usan de esta misma forma, multiplicar, restar, Etc…

- Sumar `+=`
- Restar `-=`
- Multiplicar `*=`
- Dividir `/=`
- Potencia `**=`
- Resto división `%=`
- Division sin decimal `//=`